

EDA Agent v3 Architecture Design

Overview

EDA Agent v3 represents a more advanced architecture for AI-assisted backend design and signoff workflows.

The goal is to build a **practical AI copilot for physical design, timing analysis, and signoff engineers**.

Unlike simple automation scripts, this agent integrates:

- LLM reasoning
- domain knowledge graphs
- root-cause inference
- experiment orchestration
- case memory
- multi-agent collaboration

EDA flows are expensive and high risk, therefore the system focuses on **analysis, recommendation, and controlled experimentation**.

1. System Architecture

EDA Agent v3 contains several layers.

User Interface
↓
Planner / LLM Reasoning
↓
Knowledge Graph + Case Memory
↓
Root Cause Inference Engine
↓
Primitive Execution Layer
↓
EDA Tool Adapters

High-level loop:

observe → diagnose → hypothesize → experiment → evaluate → learn

2. Multi-Agent Design

Instead of one monolithic agent, the system uses specialized agents.

Example roles:

PnR Agent

Responsible for:

- placement analysis
- congestion debugging
- CTS reasoning
- routing optimization suggestions

STA Agent

Responsible for:

- timing path analysis
- constraint validation
- slack distribution analysis
- clock skew reasoning

Signoff Agent

Responsible for:

- DRC analysis
- IR/EM hotspot reasoning
- signoff readiness checks

Experiment Agent

Responsible for:

- scheduling experiments
- running recipe variations
- comparing results

3. Knowledge Graph

EDA debugging knowledge is represented as a graph.

Nodes include:

- violation types
- physical design phenomena
- tool parameters
- design structures

Example relationships:

High Density
→ Congestion

Congestion
→ Routing Detour

Routing Detour
→ Long Net Delay

Long Net Delay
→ Setup Violation

Graph queries allow the agent to trace causal chains.

4. Root Cause Inference

The inference engine evaluates candidate causes using extracted features.

Example features:

- net length
- fanout
- congestion metrics
- clock skew
- buffer depth
- routing layer usage

Example reasoning rule:

IF
congestion > threshold
AND

net_length > threshold

THEN

root_cause = routing_detour

Multiple hypotheses are ranked based on likelihood.

5. Experiment Engine

EDA debugging frequently requires testing multiple solutions.

The experiment engine manages controlled experiments.

Example experiment:

change placement density
rerun placement optimization
compare QoR metrics

Experiment results are stored and linked to design states.

6. Recipe Search

EDA Agent v3 can explore configuration spaces automatically.

Parameters may include:

- placement density
- CTS effort
- routing effort
- buffering limits
- layer assignment preferences

Search strategies:

- heuristic search
- Bayesian optimization
- reinforcement learning

7. Case Memory

The system maintains a database of historical debugging cases.

Example structure:

Case
symptoms
root cause
actions taken
result metrics

When a new issue appears, similar cases can be retrieved.

8. Guardrails

EDA actions are expensive and risky. Guardrails restrict unsafe behavior.

Examples:

Agent cannot automatically:

- modify floorplan geometry
- add false paths
- delete design objects

High-risk actions require human approval.

9. Human-in-the-loop Workflow

Typical workflow:

1. engineer asks question
2. agent analyzes reports
3. root cause candidates generated
4. recommended experiments proposed
5. engineer approves experiment
6. agent runs experiments
7. results summarized

10. User Interface Concept

Possible interface elements:

- QoR dashboard
- violation explorer
- experiment tracker
- recommendation panel

Visualization can show:

- timing path graphs
- congestion heatmaps
- run comparison charts

11. Data Pipeline

The agent requires structured data.

Sources include:

- timing reports
- congestion maps
- DRC summaries
- power analysis reports
- routing logs
- tool run metadata

Reports are normalized into JSON schemas.

12. Future Extensions

Possible extensions:

- ML-based congestion prediction
- reinforcement learning for flow tuning
- automatic ECO generation
- layout-aware reasoning models

Conclusion

EDA Agent v3 combines:

- structured primitives
- domain knowledge graphs
- inference engines
- controlled experimentation

to create a powerful AI assistant for backend engineers and AE teams.